AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics

Md. Riaz[1*]

[1*]Department of Computer Science and Engineering, Pundra University of Science and Technology, Bogura, Bangladesh.

Corresponding author(s). E-mail(s): ;

Abstract

Natural-language analytics promises to make relational business data accessible to non-technical users, but direct text-to-SQL generation remains difficult to govern in operational reporting environments. A syntactically valid query can still violate business definitions, expose unauthorized data, or produce an unsafe report artifact. This paper presents AEGIS, a constraint-based architecture for safe LLM-assisted natural-language analytics. AEGIS restricts the language model to typed intent extraction over a governed semantic layer; deterministic components then perform coverage validation, query planning, template-based SQL compilation, post-compilation safety checking, visualization selection, and widget persistence. The approach is evaluated on a nopCommerce e-commerce deployment using static reproducible datasets: a 500-question natural-language benchmark with 425 supported and 75 realistic boundary requests, 16 source-derived nopCommerce Admin analytics oracles, 80 Admin-fidelity phrasings, and focused semantic-coverage checks. On the 500-question benchmark, AEGIS parses 498/500 prompts, answers and executes 422/425 supported requests (99.3%), and rejects or clarifies 74/75 boundary requests (98.7%). Against the Admin analytics oracles, it achieves 100.0% execution validity, 100.0% shape accuracy, and 93.8% result accuracy. The results show that AEGIS is not an unrestricted natural-language-to-SQL engine; it is a bounded architecture for safe, reusable analytics over an explicitly governed semantic layer.

Keywords: natural language interfaces text-to-SQL semantic layer dashboard generation LLM safety business intelligence constrained query generation

Md. Riaz Pundra University of Science and Technology, Bogura, Bangladesh

Natural-language analytics promises to make relational business data accessible to non-technical users, but direct text-to-SQL generation remains difficult to govern in operational reporting environments. A syntactically valid query can still violate business definitions, expose unauthorized data, or produce an unsafe report artifact. This paper presents AEGIS, a constraint-based architecture for safe LLM-assisted natural-language analytics. AEGIS restricts the language model to typed intent extraction over a governed semantic layer; deterministic components then perform coverage validation, query planning, template-based SQL compilation, post-compilation safety checking, visualization selection, and widget persistence. The approach is evaluated on a nopCommerce e-commerce deployment using static reproducible datasets: a 500-question natural-language benchmark with 425 supported and 75 realistic boundary requests, 16 source-derived nopCommerce Admin analytics oracles, 80 Admin-fidelity phrasings, and focused semantic-coverage checks. On the 500-question benchmark, AEGIS parses 498/500 prompts, answers and executes 422/425 supported requests (99.3%), and rejects or clarifies 74/75 boundary requests (98.7%). Against the Admin analytics oracles, it achieves 100.0% execution validity, 100.0% shape accuracy, and 93.8% result accuracy. The results show that AEGIS is not an unrestricted natural-language-to-SQL engine; it is a bounded architecture for safe, reusable analytics over an explicitly governed semantic layer.

1 Introduction

Organizations store much of their operational knowledge in relational databases: sales transactions, customer records, inventory, payments, shipments, refunds, and similar business data. Although this data is valuable for decision-making, access to it is uneven. Technical users can write SQL queries directly, while non-technical users often depend on developers, analysts, or administrators to prepare reports for them. This creates delay and also discourages repeated exploratory questions such as changing a date range, comparing another product group, or viewing the same metric by a different dimension.

Natural language interfaces to databases (NLIDBs) address this problem by allowing users to ask questions in ordinary language. Recent neural text-to-SQL systems and large language models have improved substantially on benchmark datasets, but practical reporting systems require more than producing a plausible SQL query. In an operational dashboard, the answer must respect business definitions, database permissions, safety constraints, and presentation requirements. A generated query that is syntactically valid can still be unsafe, semantically wrong, or unsuitable for reuse.

This paper presents **AEGIS** (Analytics Engine with Guaranteed Injection Safety), a constraint-based architecture for safe natural-language analytics. AEGIS does not ask the language model to generate SQL. Instead, the model extracts a structured analytical intent from the user's request. The rest of the pipeline maps that intent to a governed semantic layer, compiles SQL from deterministic templates, validates the query, executes it, selects a visualization, and stores the result as a reusable dashboard widget.

The central idea is to separate language understanding from query execution. The language model is useful for interpreting user wording, but it is not trusted with database structure, SQL syntax, access control, or business definitions. Those responsibilities remain in explicit system components controlled by the application owner. If a request cannot be expressed using the approved semantic layer, AEGIS returns a clarification or rejection rather than substituting the nearest available metric and silently producing a wrong answer.

This work contributes:

1. A reporting-oriented architecture that converts natural-language requests into persistent dashboard widgets rather than one-off SQL query results.
2. A governed semantic-layer contract for declaring metrics, dimensions, joins, filters, time rules, permissions, and visualization defaults.
3. A deterministic SQL compilation pipeline that generates read-only queries from approved analytical templates instead of model-authored SQL.
4. A coverage and grounding mechanism that separates answerable requests from unsupported or ambiguous requests before query compilation.
5. A reproducible nopCommerce evaluation protocol combining broad natural-language requests, Admin-oracle fidelity checks, paraphrase robustness, and semantic-coverage tests.
6. Empirical evidence that a bounded semantic-layer system can provide high supported-request execution while preserving explicit rejection behavior for realistic out-of-boundary questions.

The remainder of this paper is organized as follows. Section 2 reviews related work in natural-language interfaces, text-to-SQL, natural-language visualization, dashboard generation, and semantic layers. Section 3 presents the analytical task taxonomy that motivates the template library. Section 4 presents the AEGIS architecture. Section 5 describes the prototype implementation. Section 6 reports the evaluation. Sections 7-9 discuss implications, limitations, and conclusions.

2 Related Work

2.1 Natural Language Interfaces to Databases

Natural language database interfaces have been studied for over four decades. Early systems such as LUNAR (Woods, 1973) and TEAM (Grosz, 1983) used hand-crafted grammars and domain-specific ontologies to parse queries. These systems were brittle under vocabulary variation but established the core insight that query understanding requires a bridge between natural language and schema semantics.

NaLIR (Li and Jagadish 2014) is an important modern NLIDB because it treats ambiguity as a real problem to solve rather than an error. By showing users different possible interpretations of their question, NaLIR improves accuracy but requires the user to actively participate. AEGIS uses a similar approach - asking for clarification when the meaning is unclear - but extends it into a full widget lifecycle that NaLIR does not cover. Survey work ((Affolter et al. 2019); (Liu et al. 2026)) confirms

that ambiguity, portability, schema complexity, and controlled access remain ongoing challenges across NLIDB generations and are not solved by bigger models alone.

2.2 Neural Text-to-SQL and Benchmark Progress

The field shifted decisively toward neural approaches with Seq2SQL and WikiSQL (Zhong et al. 2018), which showed that aligned training data could teach models to produce SQL. Spider (Yu et al. 2018) advanced the challenge significantly by introducing cross-domain schemas and complex multi-table queries, becoming the standard benchmark. SParC and CoSQL (Yu et al. 2019b,a) extended the evaluation to conversational and contextual settings. BIRD (Li et al. 2023) brought benchmark queries closer to production conditions by emphasizing large databases, value grounding, and query efficiency.

Schema-aware encoding, introduced in RAT-SQL (Wang et al. 2020a), showed that explicitly modeling schema relationships improves accuracy on new databases. Constrained decoding approaches such as PICARD (Scholak et al. 2021) showed that rejecting invalid SQL tokens during generation improves results. More recent systems like G-SQL (Shalaan et al. 2025) and TriSQL (Su et al. 2026) add rule guidance and multi-stage checking. While these are impressive within the text-to-SQL area, they all focus on SQL generation quality and do not address safe data access, permission control, widget storage, or chart selection - which is what AEGIS focuses on.

2.3 Natural Language for Visualization

A parallel research stream focuses on NL-driven chart generation rather than SQL generation. nl4dv (Narechania et al. 2021) maps natural language queries to analytic tasks and visual encodings. nvBench (Luo et al. 2021) introduced a cross-domain benchmark for NL-to-visualization. Eviza (Setlur et al. 2016) enabled conversational interaction with existing visualizations. DataTone (Gao et al. 2015) managed ambiguity in NL visualization interfaces through mixed-initiative interaction, surfacing alternative chart interpretations to users - a concept AEGIS adopts in its clarification model.

2.4 Dashboard Generation

Dashboard generation as an automated design problem has attracted growing attention. DashBot (Deng et al. 2023) proposed using deep reinforcement learning to compose dashboards from a set of data insights. MultiVision (Wu et al. 2022) used bidirectional LSTM models to score individual charts and combine them into multi-view dashboards. DataShot (Wang et al. 2020b) and Calliope (Shi et al. 2021) used statistical fact extraction followed by template-based layout to generate narrative data documents.

2.5 Semantic Layers and Controlled Analytics

A semantic layer is a business-logic abstraction that maps business concepts to the actual database tables and columns. Commercial tools like dbt Metrics, Looker

Table 1 Comparative positioning of AEGIS against related systems.

System	NL Parsing	Semantic Layer	Safe SQL	Visualization	Widget Persistence	Coverage Validation	Production Evaluation
Spider / BIRD (Yu '18; Li '23)	Yes	-	-	-	-	-	Benchmark only
Seq2SQL (Zhong '18)	Yes	-	-	-	-	-	Benchmark only
RAT-SQL (Wang '20)	Yes	-	-	-	-	-	Benchmark only
PICARD (Scholak '21)	Yes	-	Partial	-	-	-	Benchmark only
NaLIR (Li '14)	Yes	-	-	-	-	-	Benchmark only
nl4dv (Narechania '21)	Yes	-	-	Yes	-	-	In-memory data
DashBot (Deng '23)	-	-	-	Yes	Partial	-	Synthetic data
Lehmann et al. (2022)	-	Yes	-	-	-	-	Position paper
AEGIS (this work)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**nopCommerc evaluation**

LookML, and Apache Superset implement semantic layers in different ways. Lehmann et al. (2022) stress the importance of controlled data access in practical NL database interfaces. Structured output enforcement for LLMs (OpenAI 2024) has been shown to improve the reliability of typed object generation, which AEGIS uses for intent extraction. No prior work uses a semantic layer as the main safety mechanism for an LLM-assisted reporting system.

2.6 Comparative Summary

3 Analytical Task Taxonomy

3.1 Taxonomy Construction

The eleven analytics primitives below were identified through a review of representative e-commerce and administrative reporting requests conducted during AEGIS design. This taxonomy is a design artifact: it defines the finite request shapes the compiler is allowed to render, rather than claiming that all possible user questions fit these shapes.

The final evaluation no longer relies on the older mixed general benchmark as the main evidence source. Instead, it uses a static nopCommerce corpus with 500 natural-language questions: 425 supported questions that should be answerable by the implemented semantic layer and 75 realistic e-commerce boundary questions that should be rejected or clarified.

Table 2 Analytical task taxonomy used by the AEGIS compiler.

Pattern	Purpose	Example
KPI / Aggregate	Single-number business summary	Total revenue this month
Ranking	Top or bottom entities by metric	Top products by revenue
Exception / Filter	Rows that meet an operational condition	Low-stock products
Trend Analysis	Metric over time	Monthly sales trend
Comparison	Metric compared across groups or periods	Revenue by order status
Summary / Group	Grouped business overview	Orders by payment status
Cohort	Population split by lifecycle stage	New versus returning customers
Funnel	Stepwise business process	Checkout progression
Correlate	Relationship between two measures	Discount versus order value
Segment	Breakdown by business dimension	Revenue by country
Tabular	Detailed record listing	Latest orders

3.2 Position of the Evaluation Corpus

The benchmark is intentionally finite. It measures whether the implemented nopCommerce semantic layer covers useful combinations of governed metrics, dimensions, time rules, predicates, and analytical patterns. It does not measure open-ended text-to-SQL capability. This matches the AEGIS claim: useful natural-language analytics should be broad within the declared semantic layer and explicit outside it.

The 500-question dataset, Admin analytics oracles, Admin-fidelity phrasings, and focused semantic-coverage checks are static artifacts so that the claims can be inspected and rerun.

4 The AEGIS System

AEGIS is designed around a simple division of responsibility. The language model interprets the user's wording, but all database-facing behavior is controlled by deterministic system components. This section describes the architecture and the main stages of the pipeline.

4.1 Design Principles

AEGIS follows five design principles.

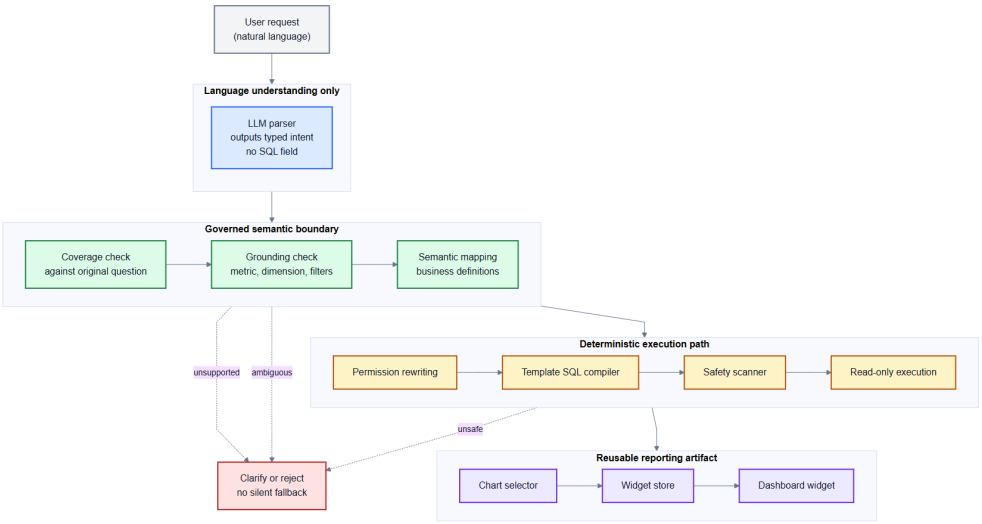

Fig. 1 AEGIS architecture pipeline from natural-language request to reusable dashboard widget.

1. **Separate understanding from execution.** The LLM is used for intent extraction, not SQL generation.
2. **Represent business meaning explicitly.** Metrics, dimensions, filters, joins, and time rules are declared in a semantic layer.
3. **Constrain query construction.** SQL is produced only by deterministic templates over approved semantic-layer objects.
4. **Treat non-answer as a valid outcome.** Unsupported or ambiguous requests should be rejected or clarified rather than forced into the nearest available query.
5. **Produce reusable analytical artifacts.** The output is not only a query result, but a dashboard widget with visualization and refresh behavior.

4.2 System Overview

The pipeline begins with a natural-language request such as "Which products brought in the most revenue this monthYes" The LLM converts this sentence into a structured intent: a ranking request, using the revenue metric, grouped by product, filtered to the current month. AEGIS then checks whether each requested concept is present in the semantic layer. If the concepts are available, the analysis planner builds a canonical plan. If a concept is missing, such as a marketing campaign dimension that has not been modeled, the request is rejected or clarified.

The compiler then converts the plan into SQL using approved templates. The LLM does not choose table names, join clauses, predicates, or SQL syntax. After compilation, a safety scanner verifies that the query is read-only and contains no forbidden constructs. The query is then executed, a visualization is selected, and the result is stored as a reusable widget.

The complete pipeline is:

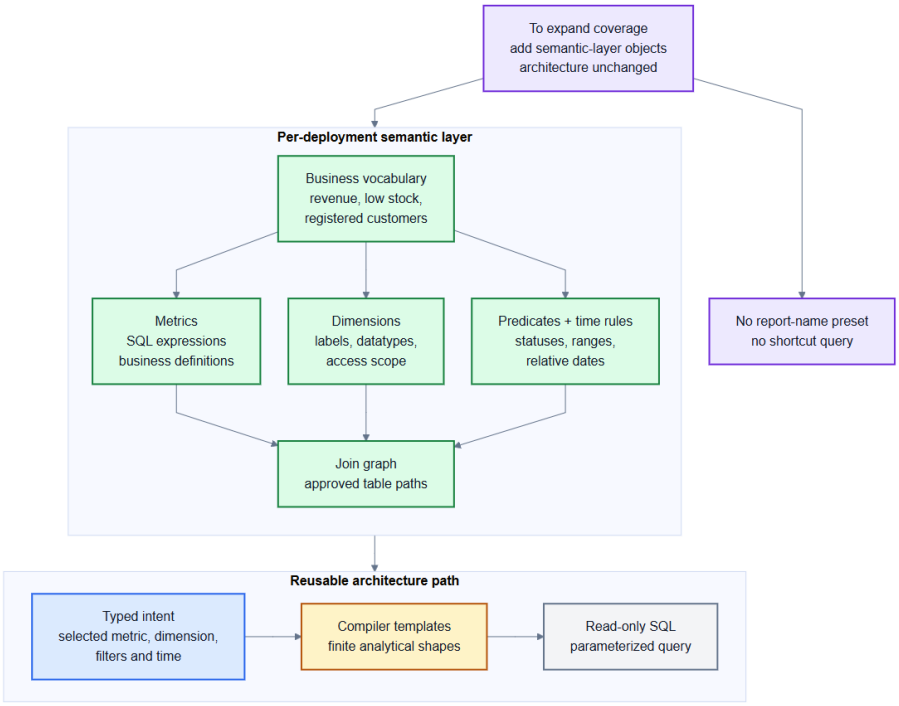

Fig. 2 Semantic-layer modularity: governed business concepts are composed before SQL is compiled.

User Request -> LLM Intent Parser -> Coverage Validator -> Semantic Mapper -> Analysis Planner -> Safe Query Compiler -> Permission Rewriter -> Query Executor -> Visualization Selector -> Widget Engine -> Dashboard

This staged design is the main difference between AEGIS and direct LLM-to-SQL systems. A direct system asks the model to produce executable SQL; AEGIS asks the model only to describe the user's analytical intent.

4.3 Semantic Layer

The semantic layer is the main governance component of AEGIS. It defines the business concepts that the system is allowed to answer. A metric specifies a measurable quantity such as revenue, order count, refund amount, or customer count. A dimension specifies how a result can be grouped or filtered, such as product, category, country, order status, payment status, or month. The semantic layer also records required joins, mandatory predicates, access rules, and default visualization choices.

This layer separates business language from the physical database schema. A user may ask for "sales", "amount spent", or "revenue", but the system maps those expressions to one approved metric definition. The same mechanism prevents unsupported concepts from being silently substituted. If the semantic layer does not define campaign attribution, review sentiment, or forecasted demand, AEGIS should not invent a query for those concepts.

4.4 Intent Parsing with Vocabulary Injection

AEGIS builds the LLM prompt from the semantic layer at runtime. The prompt lists the approved metrics and dimensions with short descriptions, and instructs the model to return a typed JSON intent object. This is called vocabulary injection. It allows the model to map flexible user wording onto approved identifiers without maintaining a separate synonym dictionary.

The parser output contains fields such as the analytical pattern, metric, dimension, time phrase, filters, sort order, and limit. For example, a request for "top products by revenue this month" should produce a ranking intent with `revenue` as the metric, `product_name` as the dimension, descending sort order, and a current-month time rule. The parser does not produce SQL.

4.5 Grounding and Coverage Analysis

The grounding stage verifies that each parsed term corresponds to an approved semantic object. It returns one of three outcomes for each important slot: resolved, ambiguous, or unsupported. This prevents silent fallback from one business concept to another.

Coverage analysis checks the original user request, not only the model's parsed output. This matters because vocabulary injection can make the model choose the nearest available identifier even when the user's actual concept is outside the semantic layer. Coverage analysis detects remaining domain terms that the semantic layer cannot explain. If a request depends on those terms, AEGIS rejects or clarifies it before any SQL is compiled.

4.6 Time Grammar and Analysis Planning

Time expressions are normalized by a dedicated time grammar. Phrases such as "today", "this month", "last 30 days", and "monthly" are converted into explicit time rules. Unsupported time phrases are reported rather than ignored. This avoids a common reporting error in which a time filter is dropped and the query silently runs over all available data.

After grounding and time normalization, AEGIS builds an analysis plan. The plan records the approved metric, dimension, filters, time rule, visualization pattern, sort order, and limit. This plan is the contract between natural-language interpretation and deterministic query compilation.

4.7 Safe Query Compiler

The compiler converts an analysis plan into SQL by expanding approved templates. It selects the required tables, resolves join paths from the semantic-layer graph, applies mandatory predicates such as soft-delete filters, binds user values as parameters, and adds grouping, ordering, and limits according to the analytical pattern.

The compiler also handles reporting-specific correctness rules. For example, an order-level metric such as total revenue cannot be grouped directly by product category without double-counting orders that contain multiple line items. In such cases,

the semantic layer can define an item-grain equivalent metric, and the planner can use that safer definition for product-level breakdowns.

The compiled SQL is then checked by a post-compilation safety scanner. Queries containing write operations, system-table access, or other forbidden constructs are rejected. The safety guarantee depends on the fact that SQL structure comes from templates and semantic-layer definitions, not from untrusted natural-language text.

4.8 Visualization and Widget Generation

Once a query executes, AEGIS selects a visualization based on the analytical pattern and result shape. Scalar results become KPI cards, ranked lists become bar charts or tables, trends become line charts, and tabular results remain tables. The widget engine stores the generated artifact so that users can refresh and reuse the report rather than asking the same question repeatedly.

4.9 Terminal Outcomes

AEGIS has three terminal outcomes:

- **ANSWER:** the request is supported, SQL is compiled, and a widget is produced.
- **CLARIFY:** the request is potentially answerable but ambiguous, so the system asks a specific follow-up question.
- **REJECT:** the request depends on concepts or operations outside the governed semantic layer.

This explicit outcome model is central to the architecture. A refusal is not treated as a crash or a missing feature when the request is genuinely outside scope; it is the correct behavior for a bounded analytical system.

5 Implementation

AEGIS is implemented as a web application with a vanilla HTML/JavaScript frontend (jQuery, Chart.js) and a Python (FastAPI) backend targeting a production nopCommerce 4.70 schema.

- **LLM Integration:** An LLM API exposed through an OpenAI-compatible `/v1/chat/completions` interface, reached through the `CUSTOM` provider profile in `aegis/server/ai_config.py` whenever `LLM_BASE_URL` is set, with `LLM_MODEL` naming the model to request. AEGIS uses structured JSON output enforcement and a system prompt constructed by injecting approved metric and dimension IDs. The reported evaluation used this OpenAI-compatible LLM API; the SQL compiler and safety layer are independent of the provider because they consume only the typed intent object.
- **Rate Limiting:** Provider-agnostic configuration module (`ai_config.py`) with sliding-window rate limiter and concurrency-safe `asyncio.Lock`.
- **Semantic Layer:** Python configuration modules containing the governed nopCommerce metrics, dimensions, predicates, and join paths used by the static evaluation

corpus. The implementation deliberately keeps synonyms out of a separate hand-maintained dictionary; wording coverage is handled through dynamic vocabulary injection over semantic-layer descriptions.

- **SQL Compiler:** Parameterized MySQL templates. BFS join path resolution across 14 tables (12 aliases). Post-compilation `_validate_sql_safety()` checks 16 forbidden patterns.
- **Visualization Selector:** Rule-based Python dictionaries. Additional rules after data: bar charts with >20 categories become tables, pie charts with >8 slices become bar charts.
- **Widget Engine:** SHA-256 plan hash deduplication. JSON file storage in prototype (designed for relational database in production).
- **Coverage Validator:** Pre-compilation gate rejects unknown metric/dimension terms with structured guidance listing available identifiers.
- **Permission Enforcement:** Permission Rewriter appends role-based WHERE predicates. Five roles: `public`, `store_manager`, `regional_manager`, `read_only`, `analyst`.

6 Evaluation

This section evaluates AEGIS on the nopCommerce e-commerce deployment using static, reproducible datasets. The evaluation separates three concerns that are often conflated in natural-language analytics: broad request coverage over the implemented semantic layer, fidelity to first-party reporting semantics where source-derived oracles exist, and correct refusal of plausible requests outside the declared semantic boundary.

6.1 Evaluation Scope and Reproducibility

The study uses one production-style schema, nopCommerce. It does not claim that AEGIS is an open-ended text-to-SQL system or that it can answer every possible e-commerce question. The tested claim is narrower: when the required business concepts are declared in the semantic layer and the required result shape is supported by deterministic compiler templates, AEGIS should parse the request, resolve it to governed concepts, compile safe SQL, execute the query, and produce an appropriate report. When a request depends on concepts outside that boundary, the correct behavior is to decline or ask for clarification rather than invent an answer.

All reported figures are backed by static datasets, benchmark scripts, and recorded result files. This matters because the evaluation includes both answerable and intentionally unsupported questions; the same corpus can be inspected and rerun rather than relying on selected examples.

6.2 Dataset and Environment

All executable evaluations run against the nopCommerce MySQL database seeded from the repository schema and mock data. The loaded database contains 1,200 customers, 2,500 orders, 6,320 order items, 1,492 shipments, 17 products, 8 categories, 8 manufacturers, and 1 store. Date-sensitive tests use the repository date-refresh script

Table 3 Evaluation corpus components.

Component	Size	Role
Expanded natural user questions	500	Main natural-language benchmark: 425 answerable questions and 75 realistic e-commerce boundary questions.
Admin fidelity phrasings	80	Five natural phrasings for each source-derived Admin fidelity target.
Admin analytics oracles	16	Source-derived nopCommerce Admin report/dashboard oracle tasks.
Focused semantic coverage	25	20 supported semantic-layer compositions and 5 boundary refusals.

so relative phrases such as "today", "this week", and "this month" remain meaningful when the benchmark is rerun.

The evaluation corpus has five static components:

For reproducibility, the released artifact includes the natural-language question set, live benchmark outputs, Admin oracle definitions, Admin benchmark outputs, semantic-coverage questions, and semantic-coverage benchmark outputs under the evaluation artifact directory.

The 500-question dataset is the main natural-language evidence. Its 425 supported questions cover KPI, ranking, trend, segmentation, listing, time-filtered requests, item-grain substitutions, customer/order/product/geography/store/status/payment/shipping dimensions, and governed predicates such as low stock. Its 75 boundary questions remain e-commerce related but require concepts not currently modeled, such as web telemetry, marketing attribution, support tickets, review-text sentiment, forecasting, churn prediction, supplier performance, fraud scoring, delivery SLA analysis, and product affinity.

6.3 Evaluation A: 500-Question Live Natural-Language Benchmark

The live benchmark sends each of the 500 static natural-language prompts through the AEGIS parser, semantic resolver, deterministic compiler, and MySQL execution path. Intent annotations are used for strict parser-slot comparison only; behavioral success is measured by whether supported questions are answered and executed and whether boundary questions are rejected or clarified.

Table 4 500-question natural-language benchmark results.

Metric	Result
Parser success	498/500 (99.6%)
Supported intent exact match	345/425 (81.2%)
Supported answer rate	422/425 (99.3%)
Supported execution validity	422/425 (99.3%)
Boundary rejection accuracy	74/75 (98.7%)

Table 5 nopCommerce Admin analytics oracle benchmark results.

Metric	Result
Execution validity	16/16 (100.0%)
Shape accuracy	16/16 (100.0%)
Result accuracy	15/16 (93.8%)

The exact-intent score is intentionally stricter than the main behavioral measures. It requires the parser to match the annotation for class, metric, dimension, time phrase, filters, sorting, and limit. The stronger evidence for the architectural claim is that 422 of 425 supported natural requests reached executable SQL, while 74 of 75 realistic but unsupported e-commerce requests were not answered as if they were in scope.

6.4 Evaluation B: nopCommerce Admin Analytics Fidelity

The Admin fidelity benchmark checks AEGIS against source-derived nopCommerce Admin analytics oracles. These are not SQL-string reproduction tests. The oracle SQL expresses the expected result according to nopCommerce source behavior, and AEGIS is scored on whether its compiled query executes, returns the expected shape, and matches the oracle result values.

This result should be read as platform-fidelity evidence, not as the whole system target. AEGIS is not intended to clone every Admin dashboard widget as a preset. The remaining mismatch is an implementation expressiveness limit: the dashboard order-average matrix requires a general multi-period matrix-summary primitive. It is not a SQL safety failure.

6.5 Evaluation C: Focused Semantic Coverage

The focused semantic coverage benchmark isolates a smaller set of hand-checked semantic-layer compositions and boundary refusals. It uses reference SQL written

Table 6 Focused semantic coverage benchmark results.

Metric	Result
Supported execution validity	20/20 (100.0%)
Supported shape accuracy	20/20 (100.0%)
Supported result accuracy	20/20 (100.0%)
Boundary rejection accuracy	5/5 (100.0%)

from the same governed nopCommerce metric and dimension definitions rather than from built-in Admin pages.

This evaluation shows value beyond Admin-report reproduction: once a business concept is modeled in the semantic layer, AEGIS can compose analytical views that do not correspond to a fixed built-in screen, while retaining refusal behavior outside the modeled boundary.

6.6 Safety Evaluation

SQL safety is enforced structurally. The LLM never emits SQL. It emits a typed intent object over injected semantic-layer vocabulary; the resolver grounds that object to approved metrics, dimensions, predicates, and time rules; the compiler emits parameterized SQL from deterministic templates; and the post-compilation safety monitor rejects forbidden constructs. Therefore the primary safety claim is architectural rather than empirical: untrusted natural-language text is not interpolated into executable SQL identifiers or clauses.

The live 500-question benchmark supports this claim operationally: 422 supported natural-language requests executed successfully through the governed compiler path, and unsupported e-commerce requests were almost always rejected or clarified rather than converted into arbitrary SQL.

6.7 Interpretation

The evaluation supports three conclusions. First, on the main 500-question natural-language dataset, AEGIS answers and executes nearly all supported nopCommerce semantic-layer requests while declining almost all realistic out-of-boundary e-commerce requests. Second, against first-party nopCommerce Admin analytics oracles, AEGIS achieves complete execution and shape validity, with result mismatches concentrated in dashboard-specific primitives not yet implemented generally. Third, the focused semantic coverage benchmark confirms that AEGIS is not merely reproducing fixed reports: the same semantic layer supports broader governed analytical combinations.

The main limitation is scope. The implementation covers a finite nopCommerce semantic layer, not every possible e-commerce concept. Adding telemetry, campaign attribution, review sentiment, forecasting, or supplier operations would require

Table 7 Structural comparison between direct LLM-to-SQL and AEGIS.

Property	Direct LLM-to-SQL	AEGIS
SQL generation	Model-generated	Template-compiled
Schema exposure to LLM	Usually required	Not required
Business definitions	Inferred from schema/context	Declared in semantic layer
Unsupported requests	May still produce SQL	Clarify or reject
Safety control	Prompting and validation	Structural constraint plus validation
Output artifact	Query/result	Saved dashboard widget

extending the semantic layer and compiler templates. That is consistent with the architecture: coverage grows by adding governed concepts and general primitives, not by allowing free-form SQL generation.

7 Discussion

The evaluation indicates that AEGIS is most useful when the goal is not unrestricted database exploration, but governed analytical reporting. In this setting, the important requirement is not only whether a system can produce a SQL query, but whether it can produce a query that respects business definitions, permissions, safety constraints, and reusable reporting workflows.

7.1 Comparison with Direct LLM-to-SQL

Direct LLM-to-SQL systems ask the model to generate executable SQL from natural language. This can be flexible, but it leaves safety and semantic correctness dependent on model behavior. AEGIS changes the role of the model. The model extracts intent, while SQL is produced by a deterministic compiler over a semantic layer.

This difference explains the main trade-off. AEGIS gives up unlimited query flexibility in exchange for stronger control over what can be asked, how it is translated, and what is allowed to execute.

7.2 Role of the Semantic Layer

The semantic layer is not only a convenience for matching words to columns. It is the boundary of the system's analytical knowledge. If a metric, dimension, or predicate is declared, users can ask for it in many natural phrasings and combine it with other supported concepts. If it is not declared, AEGIS should not invent an answer.

This design is especially important for business reporting because database column names rarely capture the full business meaning of a report. For example, revenue may need to exclude deleted orders, product-level revenue may need item-grain aggregation, and customer reports may require a specific customer identity field. These

definitions belong in the semantic layer rather than in ad hoc SQL generated per request.

7.3 Refusal as a Correct System Behavior

A central result of this work is that refusal must be measured, not hidden. In a bounded analytics system, some user questions are outside the implemented semantic layer even if they are reasonable business questions. The 500-question benchmark therefore includes realistic e-commerce boundary requests, and AEGIS is evaluated on whether it rejects or clarifies them.

This is different from treating every non-answer as a failure. For AEGIS, answering an unsupported question with a plausible but wrong query is worse than declining it. The explicit ANSWER / CLARIFY / REJECT outcome model is therefore part of the architecture, not only an error-handling feature.

7.4 Generality of the Architecture

The prototype is implemented and evaluated on nopCommerce with MySQL, but the architecture is not tied to that particular schema. To apply AEGIS to another system, the developer must define the semantic layer for that system and provide compiler templates for the target SQL dialect. The architecture remains the same: language understanding is separated from governed query compilation.

The remaining Admin fidelity mismatch illustrates this point. The missing capability is not a report-specific shortcut, but a general matrix-summary primitive. Adding such a primitive would extend the compiler's supported analytical shapes while preserving the same safety boundary.

8 Limitations and Future Work

AEGIS is intentionally bounded. Its safety and auditability come from the fact that all answerable concepts must be declared in the semantic layer and all executable SQL must be produced by deterministic compiler templates. The limitations below should therefore be read as explicit boundaries of the current nopCommerce implementation, not as reasons to bypass the architecture with free-form SQL generation.

- **Single-domain evaluation.** The final evaluation is over one e-commerce deployment, nopCommerce. The results show that the architecture works in this domain, but they do not prove cross-domain generality. Future work should repeat the same static-dataset process on a second schema such as WooCommerce or a non-commerce operational database.
- **Author-generated natural-language data.** The 500-question dataset is static and checkable, but it is still author-generated. A stronger study would collect questions from store owners or administrators, then annotate answerability and expected semantic bindings with at least two independent annotators.
- **Finite semantic coverage.** Boundary questions about web telemetry, marketing attribution, support tickets, review-text sentiment, forecasting, churn prediction, supplier performance, fraud scoring, delivery SLA analysis, and product affinity are

deliberately outside the current semantic layer. Supporting them requires adding governed metrics, dimensions, predicates, tables, and templates.

- **Remaining Admin fidelity gap.** The current Admin oracle benchmark reaches 15/16 result accuracy. The remaining dashboard mismatch requires a general multi-period matrix-summary primitive. This should be added as a reusable compiler capability, not as a hardcoded report-name preset.
- **LLM dependence for intent extraction.** The compiler and safety layer are deterministic, but natural-language intent extraction still depends on the configured LLM API. The live 500-question benchmark records parser success and exact intent agreement, but future work should compare multiple OpenAI-compatible models on the same static dataset.
- **Prototype database target.** The evaluated prototype targets nopCommerce on MySQL. This is an implementation and evaluation-scope choice, not an architectural limitation: the same semantic-layer and deterministic-compilation design can support PostgreSQL, SQL Server, or other databases by adding dialect-specific compiler templates and safety rules.
- **Widget persistence.** The prototype stores widget metadata in simple local persistence. A production deployment should move the widget registry to a transactional database with migrations, ownership policies, and administrative audit views.

9 Conclusion

AEGIS is a system for turning plain-English reporting requests into dynamic, refreshable dashboard widgets over relational databases. Its contribution has three parts.

The first is architectural. The LLM is confined to understanding the question; query construction, chart selection, and widget storage are performed by fixed templates and rules downstream of it. Because the compiler emits SQL only by expanding a closed set of templates over a curated semantic layer, and never by interpolating model-produced text, unsafe SQL is excluded by construction rather than filtered after the fact (Section 4.2, Proposition 1). This is the sense in which the design converts a probabilistic property into a structural one.

The second is a pair of mechanisms for the boundary of that vocabulary. Vocabulary injection removes the manually maintained synonym list, but in doing so makes the model structurally unable to report that a request falls outside the layer it was shown. Coverage analysis recovers that signal by running against the user's original wording rather than the model's output, and the ANSWER / CLARIFY / REJECT channel gives the pipeline somewhere to put the answer "this cannot be expressed here".

The third is evaluative. The final evaluation uses static nopCommerce datasets rather than unsupported headline claims: a 500-question natural-language benchmark, source-derived Admin analytics oracles, Admin-fidelity phrasings, and semantic-coverage checks. On the 500-question live benchmark, AEGIS answered and executed 422 of 425 supported requests and rejected or clarified 74 of 75 realistic boundary requests. Against the 16 source-derived Admin analytics oracles, it achieved 100.0%

execution validity, 100.0% shape accuracy, and 93.8% result accuracy. The remaining Admin mismatch is explicitly identified as an implementation boundary requiring a general multi-period matrix-summary primitive.

AEGIS is therefore not an infinite natural-language-to-SQL engine. It is a bounded architecture for safe natural-language analytics over a governed semantic layer, suited to environments where data privacy, consistent reporting definitions, auditability, and reusable reporting widgets matter more than unlimited query flexibility.

Declarations. **Funding** No funding was received for this work.

Conflict of interest The author declares no conflict of interest.

Data availability The evaluation datasets and result artifacts are included with the released research repository.

Code availability The prototype implementation and manuscript artifacts are maintained in the accompanying repository.

Author contribution Md. Riaz designed the architecture, implemented the prototype, constructed the evaluation datasets, ran the experiments, and wrote the manuscript.

References

Affolter K, Stockinger K, Bernstein A (2019) A comparative survey of recent natural language interfaces for databases. The VLDB Journal 28:793–819

Deng D, Wu A, Qu H, et al (2023) Dashbot: Insight-driven dashboard generation based on deep reinforcement learning. IEEE Transactions on Visualization and Computer Graphics 29(1):690–700

Gao T, Dontcheva M, Adar E, et al (2015) Datatone: Managing ambiguity in natural language interfaces for data visualization. In: Proceedings of the 28th Annual ACM Symposium on User Interface Software and Technology, pp 489–500

Li F, Jagadish HV (2014) Constructing an interactive natural language interface for relational databases. Proceedings of the VLDB Endowment 8(1):73–84

Li J, Hui B, Qu G, et al (2023) Can large language models serve as a database interface? a big bench for large-scale database grounded text-to-sqls. In: Advances in Neural Information Processing Systems

Liu M, Li J, Wang T, et al (2026) A systematic review of natural language interfaces for databases. Frontiers of Computer Science 20:2011623

Luo Y, Tang N, Li G, et al (2021) Synthesizing natural language to visualization benchmarks from nl2sql benchmarks. In: Proceedings of the ACM SIGMOD International Conference on Management of Data, pp 1235–1247

Narechania A, Srinivasan A, Stasko J (2021) nl4dv: A toolkit for generating analytic specifications for data visualization from natural language queries. IEEE

Transactions on Visualization and Computer Graphics 27(2):369–379

OpenAI (2024) Introducing structured outputs in the api

Scholak T, Schucher N, Bahdanau D (2021) Picard: Parsing incrementally for constrained auto-regressive decoding from language models. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp 9895–9901

Setlur V, Battersby SE, Tory M, et al (2016) Eviza: A natural language interface for visual analysis. In: Proceedings of the 29th Annual ACM Symposium on User Interface Software and Technology, pp 365–377

Shalaan HS, Hammad M, El-Attar NE, et al (2025) G-sql: A schema-aware and rule-guided approach for natural language to sql. IEEE Access 13:158520–158534

Shi D, Xu X, Sun F, et al (2021) Calliope: Automatic visual data story generation from a spreadsheet. IEEE Transactions on Visualization and Computer Graphics 27(2):464–474

Su X, Zhang Y, Wang X, et al (2026) A robust natural language text-to-sql generation framework. Scientific Reports 16:7892

Wang B, Shin R, Liu X, et al (2020a) Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp 7567–7578

Wang Y, Sun Z, Zhang H, et al (2020b) Datashot: Automatic generation of fact sheets from tabular data. IEEE Transactions on Visualization and Computer Graphics 26(1):895–905

Wu A, Wang Y, Zhou M, et al (2022) Multivision: Designing analytical dashboards with deep learning based recommendation. IEEE Transactions on Visualization and Computer Graphics 28(1):162–172

Yu T, Li Z, Zhang Z, et al (2018) Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp 3911–3921

Yu T, Zhang R, Er H, et al (2019a) Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, pp 1962–1979

Yu T, Zhang R, Yang K, et al (2019b) Sparc: Cross-domain semantic parsing in context. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp 4511–4523

Zhong V, Xiong C, Socher R (2018) Seq2sql: Generating structured queries from natural language using reinforcement learning. In: Proceedings of the International Conference on Learning Representations

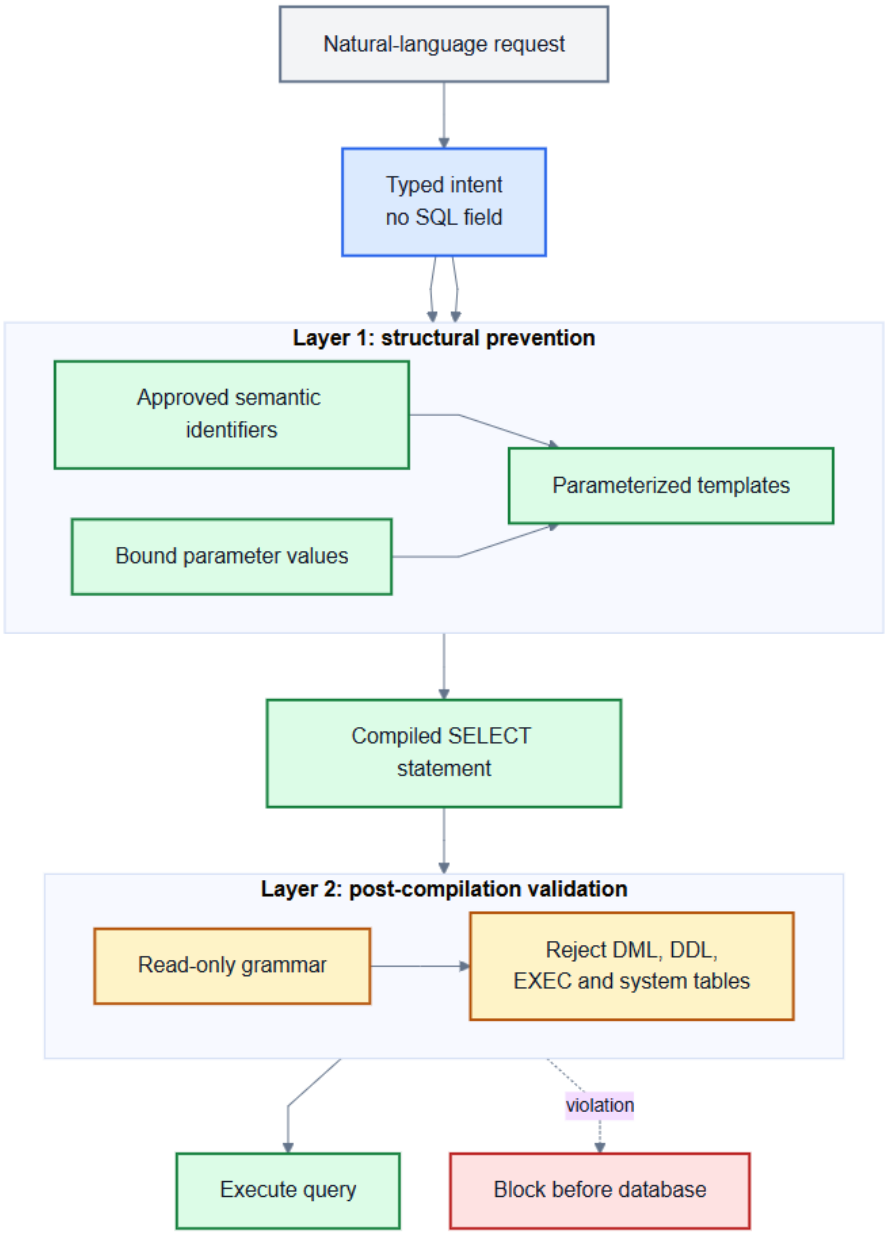

Fig. 3 Two-layer SQL safety model combining structural prevention with post-compilation validation.

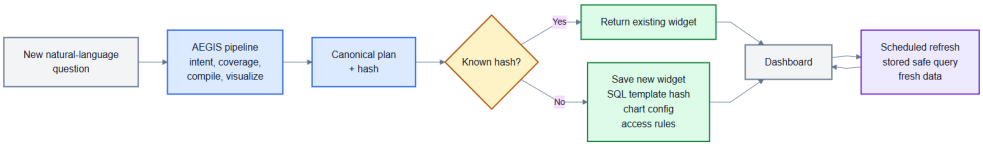

Fig. 4 Widget lifecycle for reusable natural-language analytics.